

Aula 4: Treinando a rede — risco empírico e des- cida de gradiente

Otimização • Gradientes • Funções de perda • PyTorch

Prof. Marcelo Keese Albertini

GBC073 — Inteligência Computacional • Universidade Federal de Uberlândia
2026



Intuição

Já sabemos que uma rede neural *pode representar* praticamente qualquer função (aproximação universal). A pergunta desta aula é outra: **como encontrar os pesos** que fazem a rede representar a função certa?

Plano da aula:

- ▶ Treinar = minimizar o **risco empírico** — um problema de otimização
- ▶ Ferramentas: derivadas, gradientes, Hessiana
- ▶ O algoritmo: **descida de gradiente** (à mão e com *autograd*)
- ▶ Montar o problema para redes: a rede f , as saídas, a **divergência**
- ▶ O resultado que prepara a retropropagação: $\nabla_z \text{div} = \hat{y} - y$

PARTE 1

Do risco empírico à otimização



Definição — Minimização do risco empírico (ERM)

Dado um conjunto de treino $\mathcal{D} = \{(x_1, y_1), \dots, (x_T, y_T)\}$, escolha os parâmetros que minimizam a divergência média entre a saída da rede e o alvo:

$$\hat{w} = \arg \min_w E(w), \quad E(w) = \frac{1}{T} \sum_{i=1}^T \text{div}(f(x_i; w), y_i)$$

A **perda** $E(w)$ é *empírica*: não conhecemos a distribuição real dos dados, só as amostras de $\mathcal{D}$.

Intuição

Treinar uma rede é minimizar uma função. Todo o deep learning moderno se apoia em uma única ideia: se f e div forem **diferenciáveis**, dá para minimizar $E(w)$ seguindo o gradiente.

PARTE 2

Derivadas, gradientes e mínimos



Definição — Derivada como aproximação linear

Em resolução fina o bastante, toda função suave é localmente linear:

$$f(x + \Delta x) \approx f(x) + f'(x) \Delta x$$

A derivada diz *quanto a saída muda* por unidade de mudança minúscula na entrada.

- ▶ Sinal: $f' > 0 \Rightarrow$ aumentar x aumenta f ; $f' < 0 \Rightarrow$ diminui; $f' = 0 \Rightarrow$ localmente plana. Magnitude $|f'|$: inclinação da curva

Intuição

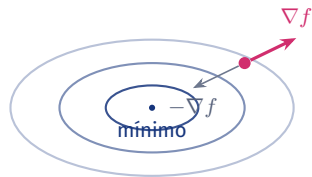
É exatamente esta leitura que o treino usa: o gradiente de E em relação a um peso diz se *aumentar aquele peso* aumenta ou diminui o erro — e o quanto.

Definição — Gradiente

Para $f : \mathbb{R}^n \rightarrow \mathbb{R}$: $\nabla_x f = \left[\frac{\partial f}{\partial x_1} \ \cdots \ \frac{\partial f}{\partial x_n} \right]^\top$, com $\Delta f \approx \nabla_x f \cdot \Delta x$

Teorema — Direção de crescimento mais rápido

O produto interno $\nabla f \cdot \Delta x$ é máximo quando Δx está *alinhado* com ∇f . Logo, o gradiente aponta para onde f **cresce mais rápido** — e $-\nabla f$ para onde decresce mais rápido.



curvas de nível: ∇f é perpendicular a elas

Teorema — Condições de mínimo local (função suave)

Em um mínimo local de $f : \mathbb{R}^n \rightarrow \mathbb{R}$ valem duas condições: $\nabla f = \mathbf{0}$ (ponto crítico) e a **Hessiana** $\nabla^2 f$, matriz das segundas derivadas $[\nabla^2 f]_{ij} = \partial^2 f / \partial x_i \partial x_j$, é **positiva semidefinida** (autovalores ≥ 0).

- ▶ Gradiente nulo sozinho não basta: máximos, mínimos, selas e inflexões *todos* têm $\nabla f = \mathbf{0}$ (em 1D: $f'' > 0$ em mínimos, $f'' < 0$ em máximos)

Cuidado

Em redes profundas ninguém verifica a Hessiana (milhões² de entradas): aceita-se o ponto onde a descida de gradiente estaciona. Em alta dimensão, pontos de **sela** são muito mais comuns que mínimos ruins.

Para discutir em aula

$f(x)$ é uma função escalar de um vetor x . Partindo de x_0 , em qual direção devemos dar um passo para obter a *maior redução* de f ?

Na direção de ∇f / Perpendicular a ∇f / Oposta a ∇f

Para discutir em aula

$f(x)$ é uma função escalar de um vetor x . Partindo de x_0 , em qual direção devemos dar um passo para obter a *maior redução* de f ?

Na direção de ∇f / Perpendicular a ∇f / Oposta a ∇f

Fechamento

Oposta ao gradiente. O gradiente é a direção de subida mais rápida; seu oposto é a descida mais rápida. Perpendicular ao gradiente o valor de f não muda (é a direção da curva de nível). Este é o coração do algoritmo da próxima seção.

PARTE 3

Descida de gradiente



Por que soluções iterativas?

Para minimizar, poderíamos resolver $\nabla_w E(w) = 0$ analiticamente...

- ▶ Funciona para funções simples (regressão linear tem solução fechada); para uma rede com milhões de pesos e não linearidades compostas, o sistema $\nabla E = 0$ é **intratável**

Intuição

A saída: começar de um *chute* $w^{(0)}$ e refiná-lo passo a passo, sempre na direção que reduz a perda. Duas decisões definem o método: *em que direção* andar e *qual o tamanho* do passo.

Cuidado

Todo o treinamento moderno — do Perceptron ao Transformer — é variante desta ideia; o que muda é como o passo é calculado (SGD, momento, Adam...).

Algoritmo — Descida de gradiente (*gradient descent*)

- ▶ Inicialize $\mathbf{w}^{(0)}$; $k \leftarrow 0$
- ▶ Enquanto não convergir: $\mathbf{w}^{(k+1)} \leftarrow \mathbf{w}^{(k)} - \eta \nabla_{\mathbf{w}} E(\mathbf{w}^{(k)})$
- ▶ Critérios de parada: $|E(\mathbf{w}^{(k+1)}) - E(\mathbf{w}^{(k)})| < \epsilon_1$ ou $\|\nabla_{\mathbf{w}} E\|^2 < \epsilon_2$

Anatomia do passo:

$$\mathbf{w} \leftarrow \mathbf{w} - \underbrace{\eta}_{\text{taxa de aprendizagem}} \cdot \underbrace{\nabla_{\mathbf{w}} E(\mathbf{w})}_{\text{gradiente da perda}}$$

- ▶ η pequena demais: converge devagar • η grande demais: oscila ou diverge
- ▶ Para funções **convexas** (formato de tigela) e η adequada, converge ao mínimo **global**; caso contrário, a um mínimo local ou sela

Minimizar $f(x) = x^2 - 4x + 5$, cujo mínimo é $x^* = 2$ (derivada $f'(x) = 2x - 4$):

```
import torch

x = torch.tensor(8.0)      # chute inicial
eta = 0.1                  # taxa de aprendizado
for t in range(60):
    grad = 2*x - 4         # derivada calculada a mao
    x = x - eta * grad     # passo: oposto ao gradiente
print(x)                  # tensor(2.0000)
```

Intuição

Cada linha do laço é a fórmula do slide anterior. O único trabalho “inteligente” foi derivar f à mão — e é exatamente isso que o *autograd* elimina.

```
x = torch.tensor(8.0, requires_grad=True)
opt = torch.optim.SGD([x], lr=0.1)

for t in range(60):
    loss = x**2 - 4*x + 5      # constroi o grafo de computacao
    opt.zero_grad()           # zera gradientes acumulados
    loss.backward()           # autograd: calcula d(loss)/dx
    opt.step()                 #  $x \leftarrow x - lr * x.grad$ 
```

- ▶ `loss.backward()` percorre o grafo de trás para frente e preenche `x.grad` — é a **retropropagação**, tema da próxima aula; `opt.step()` aplica $w \leftarrow w - \eta \nabla E$

Cuidado

Esquecer `opt.zero_grad()` é o bug clássico: o PyTorch *acumula* gradientes entre chamadas de `backward()`, e o passo passa a usar a soma de todos os gradientes já calculados.

O que a teoria garante

- ▶ Função convexa + passo adequado $\Rightarrow$ mínimo global
- ▶ Base sólida: regressão linear, logística

O que o deep learning vive

- ▶ Perda de rede profunda: **não convexa**
- ▶ Milhões de mínimos locais e selas

Intuição

Por que funciona mesmo assim? Em redes *superparametrizadas*, quase todos os mínimos locais têm perda próxima da ótima. O problema prático não é “cair no mínimo errado”, e sim caminhar de forma eficiente na paisagem — daí os otimizadores modernos.

PARTE 4

Montando o problema para redes neurais



$$\hat{w} = \arg \min_w \frac{1}{T} \sum_{i=1}^T \text{div}(f(x_i; w), y_i)$$

A rede $f(\cdot; w)$

Qual arquitetura? Quais são os parâmetros w ?

Os pares (x_i, y_i)

Como representar entradas e saídas como números?

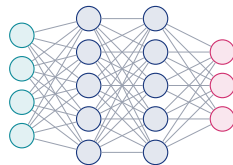
A divergência

Como medir o erro entre $\hat{y}$ e y ? Precisa ser diferenciável!

O resto da aula responde às três perguntas, uma por vez.

A rede f : o MLP e sua notação

- ▶ Rede *direta* (*feedforward*) em camadas, sem laços; a entrada é a camada 0 (sem neurônios: são os dados)
- ▶ Saída do neurônio j da camada k : $y_j^{(k)}$;
 $w_{ij}^{(k)}$: peso do neurônio i da camada $k-1$ ao neurônio j da camada k ; viés $b_j^{(k)}$
- ▶ Cada neurônio: $v = \mathbf{w}^\top \mathbf{x} + b$, $y = \varphi(v)$ — afim + ativação **diferenciável**



entrada em ciano, ocultas em azul, saída em
rosa

Intuição

Todos os pesos e vieses juntos formam o vetor gigante $\mathbf{w}$ do gradiente. Rede moderna: de milhões a trilhões de coordenadas, mesmo algoritmo.

A descida de gradiente exige derivar tudo — inclusive as ativações:

$$\sigma(z) = 1/(1 + e^{-z}), \sigma' = \sigma(1 - \sigma) \quad \tanh' = 1 - \tanh^2 \quad \text{ReLU}(z) = \max(0, z), \\ \text{ReLU}'(z) = [z > 0]$$

Histórico

Sigmoide e $\tanh$ nas ocultas dominaram até ~2012. Saturam: derivada ≈ 0 longe da origem.

Moderno

Família ReLU (ReLU, GELU, SiLU) é o padrão nas ocultas: não satura no lado positivo.

Cuidado

Ativação saturada $\Rightarrow$ gradiente $\approx 0 \Rightarrow$ o peso **para de aprender** (*desvanecimento do gradiente*) — foi o que tirou a sigmoide das ocultas; hoje ela vive quase só na saída de classificadores binários.

Definição — Softmax

Ativação que acopla todas as saídas da camada, transformando valores afins $z_1, \dots, z_K$ (os *logits*) em uma distribuição de probabilidade:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \implies \quad \hat{y}_i > 0, \quad \sum_i \hat{y}_i = 1$$

- Mudar *um* peso afeta *todas* as saídas (por isso “vetorial”); uso padrão na saída de classificadores multiclasse: $\hat{y}_i$ = probabilidade da classe i

Intuição

O softmax reaparece no coração dos Transformers: a *atenção* usa esta mesma função para converter escores de similaridade em pesos que somam 1.

- ▶ **Regressão** (saída real): use o valor diretamente; um neurônio por dimensão, saída linear
- ▶ **Binária** (“é um gato?”): alvo $y \in \{0, 1\}$, saída sigmoide $\hat{y} \in (0, 1)$ lida como $P(\text{classe} = 1)$
- ▶ **Multiclasse**: alvo em representação *one-hot*, saída softmax

Definição — Vetor one-hot

Para K classes em ordem fixa, a classe c vira o vetor com 1 na posição c e 0 nas demais. Com classes [gato, cão, camelo, chapéu, flor]:

$$\text{camelo} \mapsto \mathbf{y} = (0, 0, 1, 0, 0)^{\top}$$

- ▶ A rede produz $\hat{\mathbf{y}}$: vetor de K probabilidades que somam 1
- ▶ O ideal seria $\hat{\mathbf{y}}$ igual ao one-hot; na prática, quanto mais massa na classe certa, melhor

Classificador de dígitos (entrada $28 \times 28 = 784$ pixels, 10 classes):

```
import torch.nn as nn

modelo = nn.Sequential(
    nn.Linear(784, 256), nn.ReLU(), # camada oculta 1
    nn.Linear(256, 128), nn.ReLU(), # camada oculta 2
    nn.Linear(128, 10),          # saída: 10 logits
)

logits = modelo(x)                # x: tensor (B, 784)
```

- ▶ Cada `nn.Linear` guarda uma matriz de pesos e um viés — as coordenadas de w ; `modelo.parameters()` entrega todas ao otimizador
- ▶ Repare: **não há softmax na saída**. A rede devolve *logits* — o porquê fica claro em dois slides

PARTE 5

Divergências: medindo o erro



Definição — Divergência L2 (erro quadrático)

Para saídas reais, a escolha padrão é a distância euclidiana ao quadrado (escalada):

$$\text{div}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}\|^2 = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2 \quad \implies \quad \frac{\partial \text{div}}{\partial \hat{\mathbf{y}}} = \hat{\mathbf{y}} - \mathbf{y}$$

- Diferenciável, e a derivada é o **erro** — o $\frac{1}{2}$ só cancela o 2 da derivada. Em PyTorch: `nn.MSELoss()`

Cuidado

Para *classificação*, a L2 é ruim: com saída sigmoide/softmax gera gradientes minúsculos justo quando a rede está muito errada. Daí a próxima divergência.

Definição — Entropia cruzada (*cross-entropy*)

Para alvo one-hot $\mathbf{y}$ (classe correta c) e saída softmax $\hat{\mathbf{y}}$:

$$\text{div}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_i y_i \log \hat{y}_i = - \log \hat{y}_c \quad \frac{\partial \text{div}}{\partial \hat{y}_i} = \begin{cases} -1/\hat{y}_c & i = c \\ 0 & i \neq c \end{cases}$$

- ▶ Minimizar $-\log \hat{y}_c$ = maximizar a probabilidade da classe correta (máxima verossimilhança); para alvo one-hot, KL e entropia cruzada **coincidem**

Intuição

A entropia cruzada *explode* quando $\hat{y}_c \rightarrow 0$: gradiente enorme justo quando a rede está confiante e errada. Esse castigo acelera a convergência — é a perda de todo modelo de classificação e linguagem moderno.

Para discutir em aula

A entropia cruzada atinge seu mínimo quando $\hat{\mathbf{y}} = \mathbf{y}$. Portanto, nesse ponto, a derivada $\partial \text{div} / \partial \hat{\mathbf{y}}_c$ é zero — como acontece com a L2.

Verdadeiro / Falso

Para discutir em aula

A entropia cruzada atinge seu mínimo quando $\hat{\mathbf{y}} = \mathbf{y}$. Portanto, nesse ponto, a derivada $\partial \text{div} / \partial \hat{y}_c$ é zero — como acontece com a L2.

Verdadeiro / Falso

Fechamento

Falso. No mínimo ($\hat{y}_c = 1$) a derivada vale $-1/\hat{y}_c = -1 \neq 0$. O mínimo existe porque $\hat{y}_c$ não pode passar de 1 (restrição do softmax), não porque a curva aplaina. Consequência prática: o gradiente continua “empurrando” $\hat{y}_c$ para cima mesmo perto do acerto — em relação aos *logits*, porém, o gradiente $\hat{\mathbf{y}} - \mathbf{y}$ zera direitinho.

O resultado central da aula

O gradiente em relação aos logits é o erro

Encadeando softmax + entropia cruzada e derivando em relação aos valores afins z (os logits) que entram no softmax, quase tudo cancela:

$$\text{div} = -\log \hat{y}_c, \quad \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \quad \implies \quad \nabla_{\mathbf{z}} \text{div} = \hat{\mathbf{y}} - \mathbf{y}$$

- ▶ O mesmo vale para saída linear + L2: gradiente = $\hat{\mathbf{y}} - \mathbf{y}$
- ▶ Interpretação limpa: o que se propaga para trás a partir da saída é simplesmente o erro — probabilidade prevista menos alvo

Intuição

Não é coincidência: L2 $\leftrightarrow$ saída linear e entropia cruzada $\leftrightarrow$ softmax são pares “casados”. É o primeiro elo da cadeia que a retropropagação estende por toda a rede na próxima aula.

```
criterio = nn.CrossEntropyLoss() # log-softmax + NLL, tudo em um

logits = modelo(x)                # (B, 10) -- SEM softmax antes!
perda = criterio(logits, alvo)     # alvo: indices (B,), nao one-hot

perda.backward()                  # gradiente flui: começa em y_hat - y
```

Cuidado

Erro clássico: aplicar `nn.Softmax` na rede e usar `CrossEntropyLoss`. A perda já embute o softmax (em forma numericamente estável, via *log-sum-exp*); softmax duplo achata os gradientes e o treino “anda de lado”.

- `CrossEntropyLoss(label_smoothing=0.1)`: troca o one-hot por alvo suavizado ($1 - \epsilon$ na classe certa, $\epsilon/(K-1)$ nas demais) — truque moderno de regularização, usado em Transformers

PARTE 6

Fechamento



- ▶ Redes neurais são aproximadores universais; treiná-las é **minimizar o risco empírico** — um problema de otimização
- ▶ O **gradiente** aponta a subida mais rápida; a **descida de gradiente** caminha no sentido oposto: $w \leftarrow w - \eta \nabla_w E$
- ▶ Definimos a rede (f diferenciável), as saídas (real / sigmoide / one-hot + softmax) e a divergência (**L2; entropia cruzada**); nos pares casados, o gradiente na saída é o erro: $\nabla_z \text{div} = \hat{y} - y$

Exercício 1 — Softmax e entropia cruzada

Com logits $z = (2, 0, -1)$ e classe correta $c = 1$: (a) calcule $\hat{y} = \text{softmax}(z)$; (b) calcule a perda $-\log \hat{y}_1$; (c) calcule $\nabla_z \text{div} = \hat{y} - y$ e verifique que as componentes somam zero. Confira com *torch*.

Próxima aula: **retropropagação** — como levar $\hat{y} - y$ da saída até cada peso da rede.